

Experiment No. 26

Write a Prolog Program to implement Best First Search algorithm.

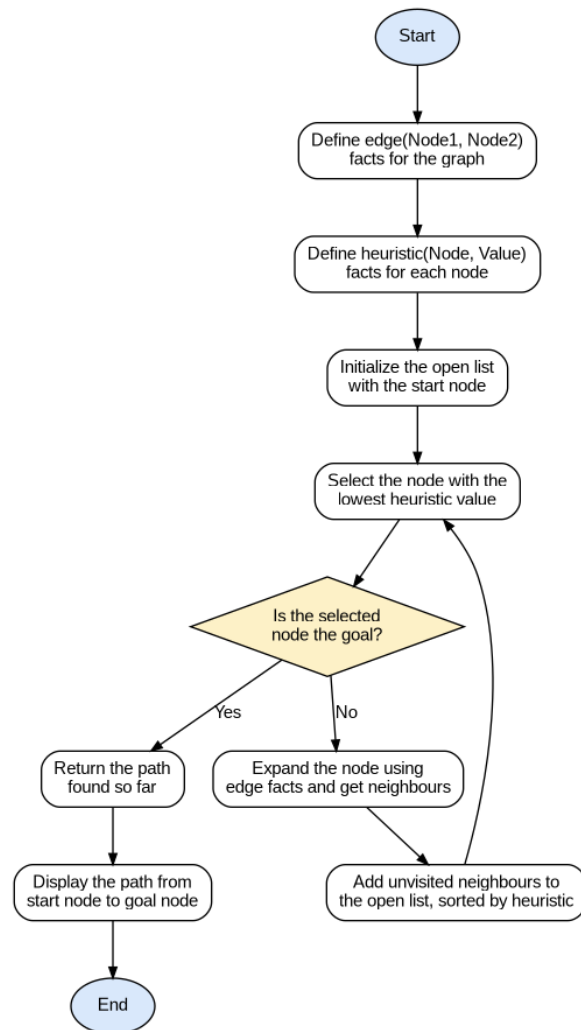
Aim

To write a Prolog program that implements the Best First Search algorithm to find a path from a start node to a goal node in a graph, using heuristic values to guide the search.

Algorithm

1. Start.
2. Define facts `edge(Node1, Node2)` to represent the connections between nodes in the graph.
3. Define facts `heuristic(Node, Value)` to represent the estimated cost from each node to the goal.
4. Represent the open list (agenda) as a list of partial paths, ordered by the heuristic value of the path's leading node.
5. Initialize the open list with a single path containing only the start node.
6. At each step, remove the path at the front of the open list, since it has the lowest heuristic value.
7. If the leading node of this path is the goal node, stop and return the path found.
8. Otherwise, expand the leading node by finding its unvisited neighbours using the edge facts.
9. Insert the newly expanded paths into the open list in sorted order according to their heuristic values.
10. Repeat steps 6 to 9 until the goal node is reached.
11. Display the resulting path from the start node to the goal node.
12. Stop.

Flowchart



Program

```

edge(a, b).
edge(a, c).
edge(b, d).
edge(c, d).
edge(d, e).
edge(b, e).

```

```

heuristic(a, 6).
heuristic(b, 4).
heuristic(c, 3).
heuristic(d, 2).
heuristic(e, 0).

```

```

path_heuristic([Node|_], H) :-
    heuristic(Node, H).

expand(Path, Children) :-
    Path = [Node|_],
    findall([Next|Path],
            ( edge(Node, Next), \+ member(Next,
Path) ),
            Children).

insert(Path, [], [Path]).
insert(Path, [P|Ps], [Path,P|Ps]) :-
    path_heuristic(Path, H1),
    path_heuristic(P, H2),
    H1 =< H2, !.
insert(Path, [P|Ps], [P|Ps1]) :-
    insert(Path, Ps, Ps1).

insert_all([], Agenda, Agenda).
insert_all([P|Ps], Agenda, Result) :-
    insert(P, Agenda, Temp),
    insert_all(Ps, Temp, Result).

bfs([Path|_], Goal, Path) :-
    Path = [Goal|_], !.
bfs([Path|Paths], Goal, Solution) :-
    expand(Path, Children),
    insert_all(Children, Paths, NewAgenda),
    bfs(NewAgenda, Goal, Solution).

best_first(Start, Goal, Path) :-
    bfs([[Start]], Goal, RevPath),
    reverse(RevPath, Path).

main :-
    best_first(a, e, Path),
    format("Path found using Best First Search: ~w~n",
[Path]).

```

Result

The program was executed successfully and the output was verified as correct.

?- main.

Path found using Best First Search: [a,c,d,e]
true.

Sample Output — Running in VS Code

Screenshot of the program being run in the Visual Studio Code integrated terminal:

```
bestfirst.pl
1  edge(a, b).
2  edge(a, c).
3  edge(b, d).
4  edge(c, d).
5  edge(d, e).
6  edge(b, e).
7
8  heuristic(a, 6).
9  heuristic(b, 4).
10 heuristic(c, 3).
11 heuristic(d, 2).
12 heuristic(e, 0).
13
14 path_heuristic([Node|_], H) :-
15     heuristic(Node, H).
16
17 expand(Path, Children) :-
18     Path = [Node|_],
19     findall([Next|Path],
20         ( edge(Node, Next), \+ member(Next, Path) ),
21         Children).
22
23 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
PS C:\Users\Athavan\AI-Lab> swipl bestfirst.pl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)
?- main.
Path found using Best First Search: [a,c,d,e]
true.
?-
UTF-8 LF Spaces: 4
```

Experiment No. 27

Write the Prolog program for Medical Diagnosis.

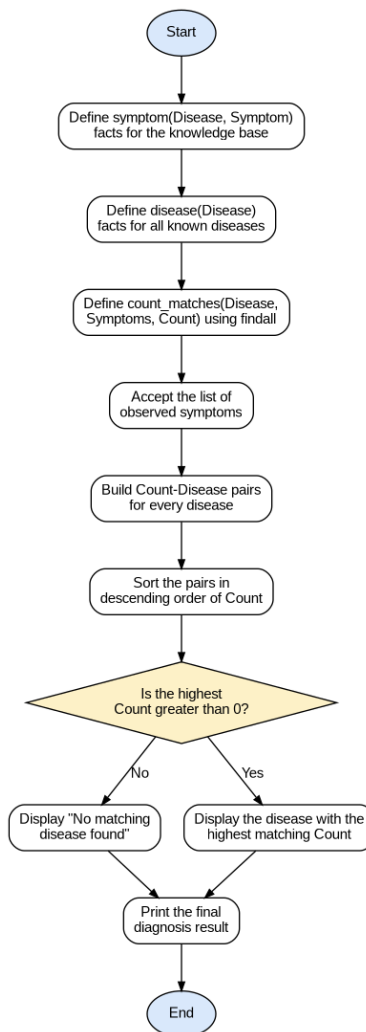
Aim

To write a Prolog program that performs a simple medical diagnosis by matching a set of observed symptoms against a knowledge base of diseases and their associated symptoms, and suggests the most likely disease.

Algorithm

1. Start.
2. Define facts `symptom(Disease, Symptom)` representing the symptoms typically associated with each disease.
3. Define facts `disease(Disease)` listing all the diseases known to the system.
4. Define the rule `count_matches(Disease, Symptoms, Count)` that uses `findall` to count how many of the given symptoms match the disease's known symptoms.
5. Define the rule `diagnose(Symptoms, BestDisease, BestCount)` that builds a list of Count-Disease pairs for every disease using `findall`.
6. Sort the list of Count-Disease pairs in descending order of match count using `sort/4`.
7. Select the disease with the highest number of matching symptoms as the most likely diagnosis.
8. If the highest count is zero, display a message indicating that no matching disease was found.
9. Accept a list of observed symptoms as input to the `diagnose` predicate.
10. Display the most likely disease along with the number of matching symptoms.
11. Stop.

Flowchart



Program

```

symptom(flu, fever).
symptom(flu, cough).
symptom(flu, body_ache).
symptom(flu, headache).

```

```

symptom(malaria, fever).
symptom(malaria, chills).
symptom(malaria, sweating).
symptom(malaria, headache).

```

```

symptom(cold, cough).
symptom(cold, sneezing).
symptom(cold, sore_throat).

```

```

symptom(typhoid, fever).
symptom(typhoid, weakness).
symptom(typhoid, stomach_pain).
symptom(typhoid, headache).

disease(flu).
disease(malaria).
disease(cold).
disease(typhoid).

count_matches(Disease, Symptoms, Count) :-
    findall(S, ( member(S, Symptoms), symptom(Disease,
S) ), Matches),
    length(Matches, Count).

diagnose(Symptoms, BestDisease, BestCount) :-
    findall(Count-Disease,
            ( disease(Disease), count_matches(Disease,
Symptoms, Count) ),
            Pairs),
    sort(0, @>=, Pairs, [BestCount-BestDisease|_]).

main :-
    Symptoms = [fever, headache, chills],
    diagnose(Symptoms, Disease, Count),
    ( Count > 0 ->
        format("Most likely diagnosis: ~w (~w matching
symptoms)~n", [Disease, Count])
        ; format("No matching disease found~n")
    ).

```

Result

The program was executed successfully and the output was verified as correct.

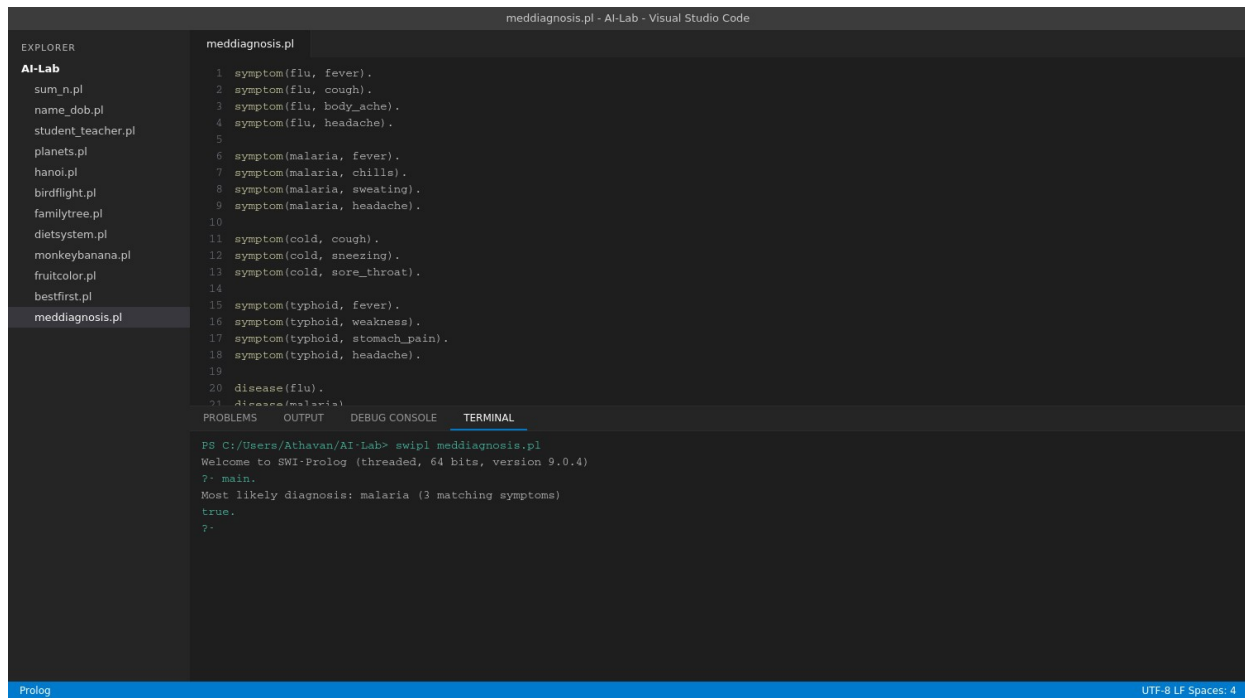
?- main.

Most likely diagnosis: malaria (3 matching symptoms)
true.

Sample Output — Running in VS Code

Screenshot of the program being run in the Visual Studio Code integrated terminal:

CSA17 - Artificial Intelligence



```
meddiagnosis.pl
1 symptom(flu, fever).
2 symptom(flu, cough).
3 symptom(flu, body_ache).
4 symptom(flu, headache).
5
6 symptom(malaria, fever).
7 symptom(malaria, chills).
8 symptom(malaria, sweating).
9 symptom(malaria, headache).
10
11 symptom(cold, cough).
12 symptom(cold, sneezing).
13 symptom(cold, sore_throat).
14
15 symptom(typhoid, fever).
16 symptom(typhoid, weakness).
17 symptom(typhoid, stomach_pain).
18 symptom(typhoid, headache).
19
20 disease(flu).
21 disease(malaria).
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL**

```
P8 C:/Users/Athavan/AI-Lab> swipl meddiagnosis.pl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)
?- main.
Most likely diagnosis: malaria (3 matching symptoms)
true.
?-
```

Prolog UTF-8 LF Spaces: 4

Experiment No. 28

Write a Prolog Program for forward Chaining. Incorporate required queries.

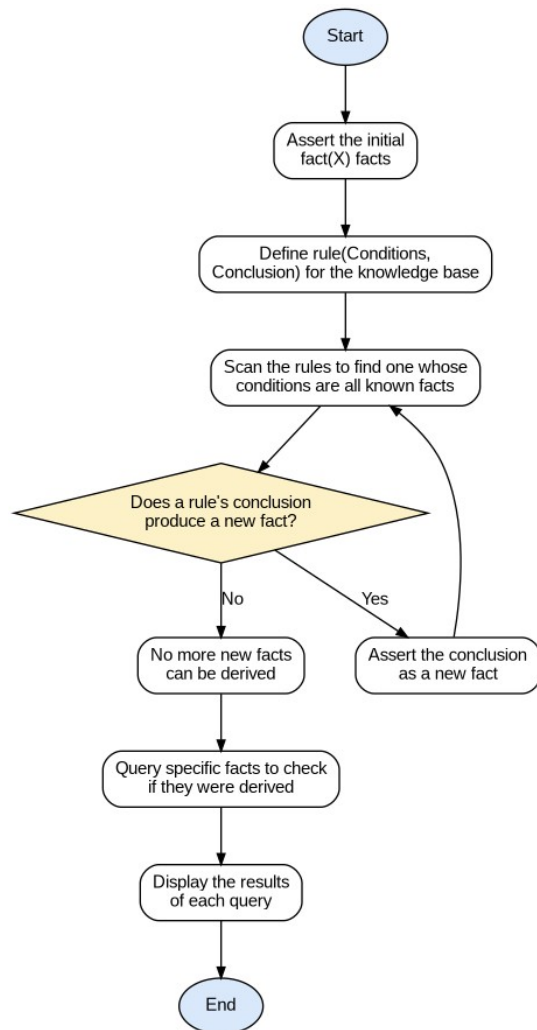
Aim

To write a Prolog program that implements forward chaining, where new facts are derived from existing facts and rules until no more new facts can be inferred, and to incorporate queries to test the derived facts.

Algorithm

1. Start.
2. Declare fact/1 as dynamic and define the initial known facts.
3. Define the rules of the knowledge base in the form rule(Conditions, Conclusion), where Conditions is a list of sub-goals.
4. Define the predicate all_true(Conditions) that succeeds if every condition in the list is already a known fact.
5. Define the predicate forward_chain that searches for a rule whose Conclusion is not yet a known fact but whose Conditions are all satisfied.
6. If such a rule is found, assert the Conclusion as a new fact using assertz and display a message.
7. Recursively call forward_chain again so that newly asserted facts can trigger further rules.
8. When no rule can produce a new fact, allow forward_chain to succeed with no further action.
9. Define the predicate query(X) that checks whether X is a known fact after forward chaining.
10. Run forward_chain to derive all possible new facts from the initial facts.
11. Query specific facts to check whether they have been successfully derived.
12. Stop.

Flowchart



Program

```

:- dynamic(fact/1).

fact(has_feathers).
fact(lays_eggs).
fact(can_fly).

rule([has_feathers, lays_eggs], is_bird).
rule([is_bird, can_fly], is_flying_bird).
rule([is_flying_bird], can_migrate).

all_true([]).
all_true([C|Cs]) :-
    fact(C),

```

```

    all_true(Cs).

forward_chain :-
    rule(Conditions, Conclusion),
    \+ fact(Conclusion),
    all_true(Conditions),
    assertz(fact(Conclusion)),
    format("Derived new fact: ~w~n", [Conclusion]),
    forward_chain.
forward_chain.

query(X) :-
    ( fact(X) ->
        format("~w is known to be true~n", [X])
    ; format("~w cannot be derived~n", [X])
    ).

main :-
    forward_chain,
    query(is_bird),
    query(is_flying_bird),
    query(can_migrate),
    query(can_swim).

```

Result

The program was executed successfully and the output was verified as correct.

```
?- main.
```

```

Derived new fact: is_bird
Derived new fact: is_flying_bird
Derived new fact: can_migrate
is_bird is known to be true
is_flying_bird is known to be true
can_migrate is known to be true
can_swim cannot be derived
true.

```

Sample Output — Running in VS Code

Screenshot of the program being run in the Visual Studio Code integrated terminal:

forwardchain.pl - AI-Lab - Visual Studio Code

EXPLORER

AI-Lab

sum_n.pl
name_dob.pl
student_teacher.pl
planets.pl
hanoi.pl
birdflight.pl
familytree.pl
dietsystem.pl
monkeybanana.pl
fruitcolor.pl
bestfirst.pl
meddiagnosis.pl
forwardchain.pl

forwardchain.pl

```

1  :- dynamic(fact/1).
2
3  fact(has_feathers).
4  fact(lays_eggs).
5  fact(can_fly).
6
7  rule([has_feathers, lays_eggs], is_bird).
8  rule([is_bird, can_fly], is_flying_bird).
9  rule([is_flying_bird], can_migrate).
10
11 all_true([]).
12 all_true([C|Cs]) :-
13     fact(C),
14     all_true(Cs).
15
16 forward_chain :-
17     rule(Conditions, Conclusion),
18     \+ fact(Conclusion),
19     all_true(Conditions),
20     assertz(fact(Conclusion)),
21     format("#Derived new fact: %s\n", [Conclusion]).

```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

```

PS C:/Users/Athavan/AI-Lab> swipl forwardchain.pl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)
?- main.
Derived new fact: is_bird
Derived new fact: is_flying_bird
Derived new fact: can_migrate
is_bird is known to be true
is_flying_bird is known to be true
can_migrate is known to be true
can_swim cannot be derived
true.
?-

```

Prolog UTF-8 LF Spaces: 4

Page 4

Experiment No. 29

Write a Prolog Program for backward Chaining. Incorporate required queries.

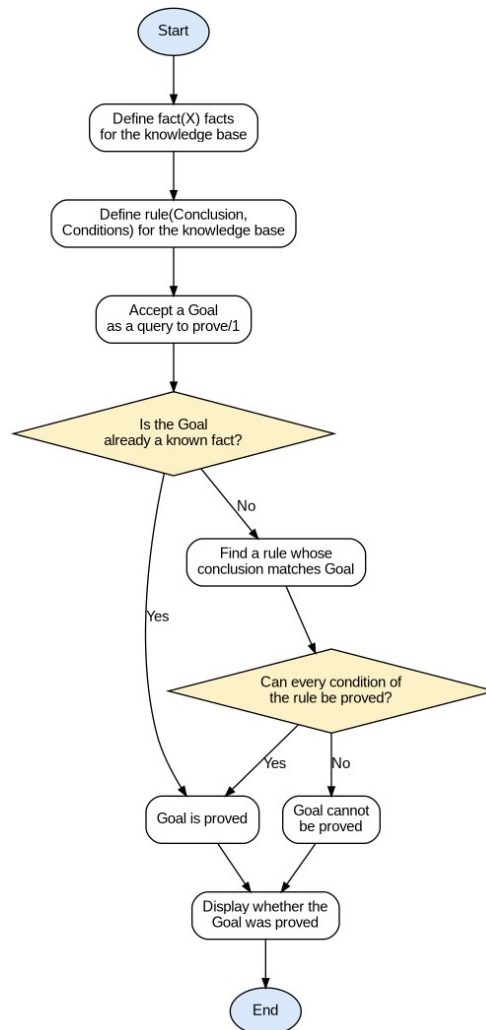
Aim

To write a Prolog program that implements backward chaining, where a goal is proved by recursively working backwards through rules to check whether its supporting conditions can be satisfied by known facts, and to incorporate queries to test different goals.

Algorithm

1. Start.
2. Define the known facts using a `fact(X)` predicate representing basic true statements in the knowledge base.
3. Define the rules of the knowledge base in the form `rule(Conclusion, Conditions)`, where `Conditions` is a list of sub-goals required to prove the `Conclusion`.
4. Define the predicate `prove(Goal)` that succeeds immediately if `Goal` is already a known fact.
5. If `Goal` is not directly a known fact, find a rule whose conclusion matches `Goal`.
6. Define the predicate `prove_all(Conditions)` that recursively calls `prove` on each condition in the list.
7. If all conditions of the matched rule can be proved, then the original `Goal` is also proved.
8. If no rule matches or a condition cannot be proved, the `prove` predicate fails for that `Goal`.
9. Define the predicate `query(Goal)` that reports whether `Goal` could be proved.
10. Accept different goals as queries to the `prove` predicate.
11. Display whether each goal was successfully proved or not.
12. Stop.

Flowchart



Program

```

fact(has_feathers).
fact(lays_eggs).
fact(can_fly).

rule(is_bird, [has_feathers, lays_eggs]).
rule(is_flying_bird, [is_bird, can_fly]).
rule(can_migrate, [is_flying_bird]).
rule(can_swim, [is_water_bird]).

prove(Goal) :-
    fact(Goal), !.
prove(Goal) :-
    rule(Goal, Conditions),

```

```

    prove_all(Conditions).

prove_all([]).
prove_all([C|Cs]) :-
    prove(C),
    prove_all(Cs).

query(Goal) :-
    ( prove(Goal) ->
      format("~w can be proved true~n", [Goal])
    ; format("~w cannot be proved~n", [Goal])
    ).

main :-
    query(is_bird),
    query(is_flying_bird),
    query(can_migrate),
    query(can_swim).

```

Result

The program was executed successfully and the output was verified as correct.

```

?- main.
is_bird can be proved true
is_flying_bird can be proved true
can_migrate can be proved true
can_swim cannot be proved
true.

```

Sample Output — Running in VS Code

Screenshot of the program being run in the Visual Studio Code integrated terminal:

CSA17 - Artificial Intelligence

```
backwardchain.pl
1 fact(has_feathers).
2 fact(lays_eggs).
3 fact(can_fly).
4
5 rule(is_bird, [has_feathers, lays_eggs]).
6 rule(is_flying_bird, [is_bird, can_fly]).
7 rule(can_migrate, [is_flying_bird]).
8 rule(can_swim, [is_water_bird]).
9
10 prove(Goal) :-
11     fact(Goal), !.
12 prove(Goal) :-
13     rule(Goal, Conditions),
14     prove_all(Conditions).
15
16 prove_all([]).
17 prove_all([C(Cs)] :-
18     prove(C).
19     prove_all(Cs).
20
21 query(Goal) :-
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL**

```
P8 C:/Users/Athavan/AI-Lab> swipl backwardchain.pl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)
?- main.
is_bird can be proved true
is_flying_bird can be proved true
can_migrate can be proved true
can_swim cannot be proved
true.
?-
```

Prolog UTF-8 LF Spaces: 4

Experiment No. 30

Create a Web Blog using WordPress to demonstrate Anchor Tag, Title Tag, etc.

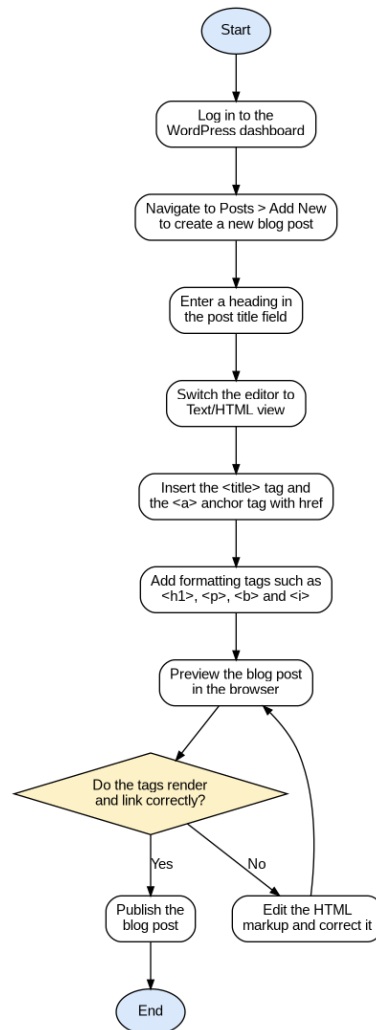
Aim

To create a web blog using WordPress and to demonstrate the use of HTML tags such as the Title Tag and the Anchor Tag, along with other basic formatting tags, within a published blog post.

Algorithm

1. Start.
2. Log in to the WordPress dashboard using valid administrator credentials.
3. Navigate to Posts > Add New to create a new blog post.
4. Enter a suitable heading for the blog post in the post title field.
5. Switch the post editor to the Text/HTML view so that raw HTML markup can be entered.
6. Use the <title> tag to set the text that appears on the browser tab for the page.
7. Use the <a> (anchor) tag with the href attribute to create a hyperlink to another web page.
8. Use the target="_blank" attribute of the anchor tag so that the linked page opens in a new browser tab.
9. Add additional formatting tags such as <h1>, <p>, and <i> to demonstrate other basic HTML elements within the post.
10. Preview the blog post to verify that the anchor tag correctly navigates to the linked page and the title tag is reflected on the browser tab.
11. If the tags do not render or link correctly, edit the HTML markup and preview again.
12. Publish the blog post once it renders correctly.
13. Stop.

Flowchart



Program

```

<!DOCTYPE html>
<html>
<head>
<title>My First WordPress Blog - AI Lab</title>
</head>
<body>

<h1>Welcome to My AI Lab Blog</h1>

<p>This blog post demonstrates the use of basic HTML
tags
inside a WordPress post, including the anchor tag and the

```

```
title tag.</p>
```

```
<p>Click the link below to visit the official WordPress
website:</p>
```

```
<a href="https://wordpress.org" target="_blank">
Visit WordPress.org
</a>
```

```
<p>This paragraph is shown in <b>bold</b> and
<i>italic</i> to illustrate simple text formatting
tags.</p>
```

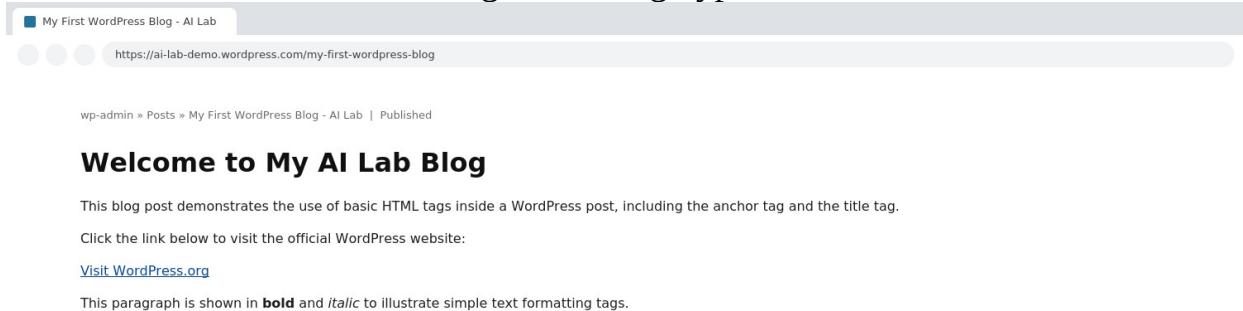
```
</body>
</html>
```

Result

The blog post was published successfully in WordPress. The browser tab displayed the text set in the <title> tag, and clicking the anchor tag opened <https://wordpress.org> in a new browser tab as expected.

Sample Output — Rendered in Browser

Screenshot of the published blog post as rendered in the browser, showing the browser-tab title and the working anchor-tag hyperlink:



Experiment No. 31

Write a Prolog program to implement pattern matching.

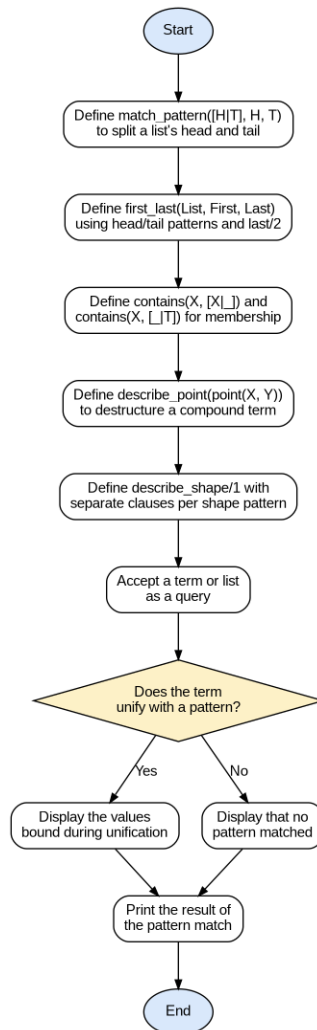
Aim

To write a Prolog program that demonstrates pattern matching by unifying different terms, lists, and compound structures against defined patterns, and to test the matching using various queries.

Algorithm

1. Start.
2. Define the rule `match_pattern([H|T], H, T)` that splits a list into its head and tail directly through pattern matching in the clause head.
3. Define the rule `first_last(List, First, Last)` that matches the first element using `[First|Rest]` and finds the last element using the built-in `last/2` predicate.
4. Define the rule `contains(X, [X|_])` that succeeds immediately when `X` matches the head of the list, using a cut to stop further search.
5. Define a second clause `contains(X, [_|T]) :- contains(X, T)` that recursively searches the tail of the list when the head does not match.
6. Define the rule `describe_point(point(X, Y))` that destructures a compound term to extract its coordinates.
7. Define multiple clauses of `describe_shape/1`, each matching a different compound term pattern such as `circle(R)` or `rectangle(W, H)`.
8. Define a final catch-all clause of `describe_shape/1` that matches any term not covered by the earlier patterns.
9. Accept different terms and lists as queries to test the pattern-matching rules.
10. If unification succeeds, display the values that were bound during the match.
11. If no defined pattern matches, display an appropriate message.
12. Stop.

Flowchart



Program

```

match_pattern([H|T], H, T).

first_last([First|Rest], First, Last) :-
    last(Rest, Last).
first_last([Only], Only, Only).

contains(X, [X|_]) :- !.
contains(X, [_|T]) :-
    contains(X, T).

describe_point(point(X, Y)) :-
    format("Point matched with X = ~w and Y = ~w~n",
[X, Y]).
  
```

```

describe_shape(circle(R)) :-
    format("Circle matched with radius = ~w~n", [R]).
describe_shape(rectangle(W, H)) :-
    format("Rectangle matched with width = ~w and
height = ~w~n", [W, H]).
describe_shape(_) :-
    format("Shape did not match any known pattern~n").

main :-
    List = [a, b, c, d],
    match_pattern(List, Head, Tail),
    format("List ~w matched as Head = ~w, Tail = ~w~n",
[List, Head, Tail]),

    first_last([1,2,3,4,5], First, Last),
    format("First = ~w, Last = ~w~n", [First, Last]),

    ( contains(c, List) ->
        format("c was found in the list~n")
    ; format("c was not found in the list~n")
    ),

    describe_point(point(3, 7)),

    describe_shape(circle(5)),
    describe_shape(rectangle(4, 6)),
    describe_shape(triangle(3,4,5)).

```

Result

The program was executed successfully and the output was verified as correct.

?- main.

List [a,b,c,d] matched as Head = a, Tail = [b,c,d]

First = 1, Last = 5

c was found in the list

Point matched with X = 3 and Y = 7

Circle matched with radius = 5

Rectangle matched with width = 4 and height = 6

Shape did not match any known pattern

true.

Sample Output — Running in VS Code

Screenshot of the program being run in the Visual Studio Code integrated terminal:

The screenshot shows the Visual Studio Code interface with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a directory named 'AI-Lab' containing several Prolog files, with 'patternmatch.pl' selected. The code editor displays the contents of 'patternmatch.pl', which defines a set of Prolog rules for pattern matching. The terminal shows the execution of the program, with the SWI-Prolog interpreter outputting the results of the queries.

```

patternmatch.pl
1 match_pattern([H|T], H, T).
2
3 first_last([First|Rest], First, Last) :-
4     last(Rest, Last).
5 first_last([Only], Only, Only).
6
7 contains(X, [X|_]) :- !.
8 contains(X, [_|T]) :-
9     contains(X, T).
10
11 describe_point(point(X, Y)) :-
12     format("Point matched with X = ~w and Y = ~w~n", [X, Y]).
13
14 describe_shape(circle(R)) :-
15     format("Circle matched with radius = ~w~n", [R]).
16 describe_shape(rectangle(W, H)) :-
17     format("Rectangle matched with width = ~w and height = ~w~n", [W, H]).
18 describe_shape(_) :-
19     format("Shape did not match any known pattern~n").
20
21 :- main :-
22
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
PS C:\Users\Athavan\AI-Lab> swipl patternmatch.pl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)
?- main.
List [a,b,c,d] matched as Head = a, Tail = [b,c,d]
First = 1, Last = 5
c was found in the list
Point matched with X = 3 and Y = 7
Circle matched with radius = 5
Rectangle matched with width = 4 and height = 6
Shape did not match any known pattern
true.
?-
  
```

Experiment No. 32

Write a Prolog program to find the number of vowels.

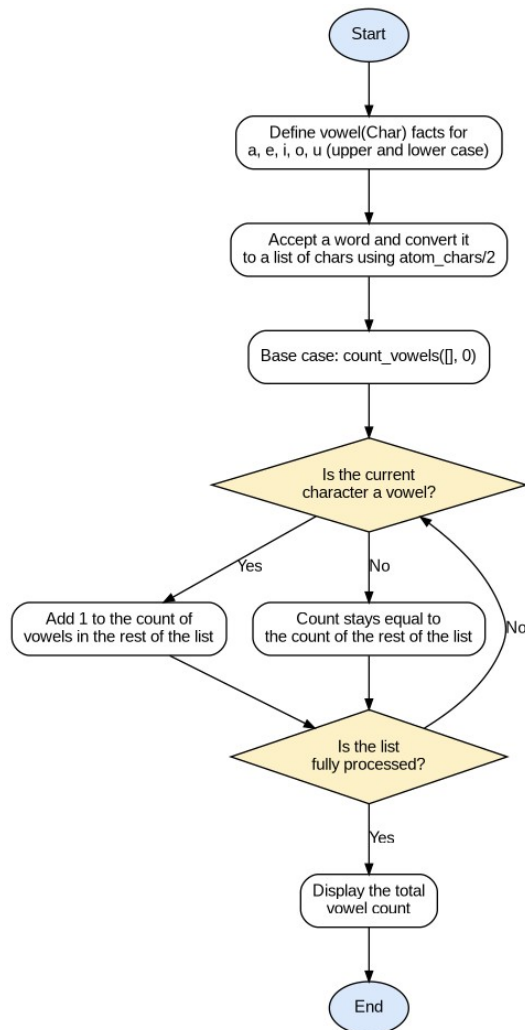
Aim

To write a Prolog program that counts the number of vowels present in a given word by checking each character of the word against the set of vowel characters.

Algorithm

1. Start.
2. Define facts `vowel(Char)` for each vowel character, including both lowercase and uppercase forms.
3. Define the base case `count_vowels([], 0)`, meaning an empty list of characters has zero vowels.
4. Define the recursive clause `count_vowels([H|T], Count)` that checks whether H is a vowel using the `vowel/1` facts, cutting once a match is found.
5. If H is a vowel, recursively count the vowels in the remaining list T and add 1 to obtain Count.
6. Define a second recursive clause that, when H is not a vowel, sets Count equal to the vowel count of the remaining list T.
7. Define the rule `vowel_count(Word, Count)` that converts Word into a list of characters using `atom_chars/2` and calls `count_vowels` on that list.
8. Accept a word as a query to `vowel_count`.
9. Display the total number of vowels found in the word.
10. Repeat the process for multiple words, including one with no vowels, using separate queries.
11. Stop.

Flowchart



Program

```
vowel(a). vowel(e). vowel(i). vowel(o). vowel(u).
vowel('A'). vowel('E'). vowel('I'). vowel('O').
vowel('U').
```

```
count_vowels([], 0).
count_vowels([H|T], Count) :-
    vowel(H), !,
    count_vowels(T, RestCount),
    Count is RestCount + 1.
count_vowels([_|T], Count) :-
    count_vowels(T, Count).
```

```
vowel_count(Word, Count) :-
```

```
atom_chars(Word, Chars),
count_vowels(Chars, Count).
```

main :-

```
vowel_count(prolog, C1),
format("Number of vowels in 'prolog': ~w~n", [C1]),
vowel_count('Artificial Intelligence', C2),
format("Number of vowels in 'Artificial
Intelligence': ~w~n", [C2]),
vowel_count(rhythm, C3),
format("Number of vowels in 'rhythm': ~w~n", [C3]).
```

Result

The program was executed successfully and the output was verified as correct.

?- main.

Number of vowels in 'prolog': 2

Number of vowels in 'Artificial Intelligence': 10

Number of vowels in 'rhythm': 0

true.

Sample Output — Running in VS Code

Screenshot of the program being run in the Visual Studio Code integrated terminal:

```
vowelcount.pl - AI-Lab - Visual Studio Code

EXPLORER
AI-Lab
sum_n.pl
name_dob.pl
student_teacher.pl
planets.pl
hanoi.pl
birdflight.pl
familytree.pl
dietsystem.pl
monkeybanana.pl
fruitcolor.pl
bestfirst.pl
meddiagnosis.pl
forwardchain.pl
backwardchain.pl
patternmatch.pl
vowelcount.pl

1 vowel(a). vowel(e). vowel(i). vowel(o). vowel(u).
2 vowel('A'). vowel('E'). vowel('I'). vowel('O'). vowel('U').
3
4 count_vowels([], 0).
5 count_vowels([_|T], Count) :-
6     vowel(H), !,
7     count_vowels(T, RestCount),
8     Count is RestCount + 1.
9 count_vowels([_|T], Count) :-
10    count_vowels(T, Count).
11
12 vowel_count(Word, Count) :-
13     atom_chars(Word, Chars),
14     count_vowels(Chars, Count).
15
16 main :-
17     vowel_count(prolog, C1),
18     format("Number of vowels in 'prolog': ~w~n", [C1]),
19     vowel_count('Artificial Intelligence', C2),
20     format("Number of vowels in 'Artificial Intelligence': ~w~n", [C2]),
21     vowel_count(rhythm, C3),
22     format("Number of vowels in 'rhythm': ~w~n", [C3]),
23     !.
24
25 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
PS C:/Users/Athavan/AI-Lab> swipl vowelcount.pl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)
?- main.
Number of vowels in 'prolog': 2
Number of vowels in 'Artificial Intelligence': 10
Number of vowels in 'rhythm': 0
true.
?- 
```